

Deduplizierung und Aggregationen

Section 3 · Data Transformation and Modelling · Quelle: PySpark-/Databricks-Referenzdokumentation

Rohdaten enthalten in der Praxis fast immer Duplikate – sei es durch doppelte Quellsystem-Exporte, wiederholte Streaming-Ereignisse oder fehlerhafte Uploads. Der Exam Guide verlangt sowohl die Beherrschung von Deduplizierungs-Operationen als auch der klassischen Aggregationsfunktionen (`count`, `approx_count_distinct`, Mittelwert, `summary`). Als Beispieldaten dient eine kleine Kundentabelle mit absichtlich eingebauten Duplikaten:

```
kunden = spark.createDataFrame([
    (1, "Anna", "Berlin", "anna@mail.de"),
    (2, "Ben", "München", "ben@mail.de"),
    (2, "Ben", "München", "ben@mail.de"), # exaktes Duplikat von Zeile 2
    (3, "Clara", "Hamburg", "clara@mail.de"),
    (3, "Clara", "Hamburg", "clara2@mail.de"), # gleiche ID, andere E-Mail
], ["customer_id", "name", "city", "email"])

kunden.createOrReplaceTempView("kunden")
```

1. Deduplizierung: `distinct()` und `dropDuplicates()`

`distinct()` entfernt Zeilen, die in *allen* Spalten exakt übereinstimmen. Im Beispiel wird dadurch nur die exakte Kopie von Ben (Zeile 3) entfernt – die beiden Clara-Zeilen unterscheiden sich in der E-Mail-Adresse und bleiben daher beide erhalten.

```
kunden_distinct = kunden.distinct() # 4 statt 5 Zeilen
```

```
SELECT DISTINCT * FROM kunden;
```

`dropDuplicates()` (ohne Argumente identisch zu `distinct()`) erlaubt zusätzlich, Duplikate nur anhand einer *Teilmenge* von Spalten zu bestimmen. Im Beispiel werden nun beide Clara-Zeilen als Duplikate behandelt, da nur `customer_id` verglichen wird – welche der beiden Varianten (mit welcher E-Mail-Adresse) behalten wird, ist dabei nicht deterministisch garantiert.

```
kunden_ohne_duplikate = kunden.dropDuplicates(["customer_id"]) # 3 Zeilen: 1, 2, 3
kunden_exakt = kunden.dropDuplicates() # ohne Spaltenliste: identisch zu distinct()
```

```
SELECT customer_id, FIRST(name) AS name, FIRST(city) AS city, FIRST(email) AS email
FROM kunden
GROUP BY customer_id;
-- SQL kennt keine direkte Entsprechung zu dropDuplicates(subset); GROUP BY mit
-- Aggregatfunktionen wie FIRST()/MAX() bildet das Verhalten nach.
```

2. Zählen: count() und approx_count_distinct()

`count()` als Aktion auf einem DataFrame liefert die exakte Zeilenzahl; als Aggregatfunktion innerhalb von `agg()` oder `groupBy().agg()` zählt sie Werte pro Gruppe. Für die Anzahl *unterschiedlicher* Werte einer Spalte gibt es zwei Optionen: das exakte, aber bei großen Datenmengen teure `countDistinct()` sowie das deutlich günstigere, approximative `approx_count_distinct()` (basierend auf dem HyperLogLog++-Algorithmus).

```
from pyspark.sql.functions import count, countDistinct, approx_count_distinct

kunden.count() # exakte Gesamtzeilenzahl (5)
kunden.select(countDistinct("customer_id")).show() # exakt: 3
kunden.select(approx_count_distinct("customer_id")).show() # approximativ, mit
Standardfehler 5%

# Standardfehler (rsd) explizit einstellen, z.B. für höhere Genauigkeit auf Kosten der
Performance:
kunden.select(approx_count_distinct("customer_id", 0.01)).show()
```

```
SELECT
  COUNT(*) AS anzahl_zeilen,
  COUNT(DISTINCT customer_id) AS exakte_anzahl_kunden,
  APPROX_COUNT_DISTINCT(customer_id) AS approx_anzahl_kunden
FROM kunden;
```

`approx_count_distinct()` lohnt sich vor allem bei sehr großen Tabellen mit Millionen unterschiedlicher Werte (z. B. eindeutige Besucher-IDs auf einer Webseite): Der exakte `COUNT(DISTINCT ...)` erzwingt einen vollständigen Shuffle aller Werte, während die approximative Variante mit konstantem, deutlich geringerem Speicherbedarf auskommt – auf Kosten einer kleinen, konfigurierbaren Fehlertoleranz.

3. Aggregationen: Mittelwert, `groupBy().agg()` und `Summary`

Für Kennzahlen pro Gruppe wird typischerweise `groupBy()` mit anschließendem `agg()` kombiniert. Die Funktionen `avg()` und `mean()` sind dabei reine Aliase und vollständig austauschbar. Als zweites Beispiel dient eine kleine Bestelltabelle:

```
bestellungen = spark.createDataFrame([
    (1, "Anna", 49.90),
    (2, "Anna", 12.50),
    (3, "Ben", 99.00),
    (4, "Clara", 20.00),
    (5, "Clara", 35.00),
], ["order_id", "customer_name", "amount"])
bestellungen.createOrReplaceTempView("bestellungen")
```

```
from pyspark.sql.functions import avg, mean, min as spark_min, max as spark_max

kunden.groupBy("city").agg(
    count("*").alias("anzahl_kunden"),
    approx_count_distinct("customer_id").alias("anzahl_eindeutig"),
).show()

bestellungen.groupBy("customer_name").agg(
    avg("amount").alias("durchschnittsbetrag"), # avg() und mean() sind identisch
    spark_min("amount").alias("min_betrag"),
    spark_max("amount").alias("max_betrag"),
).show()
```

```
SELECT city, COUNT(*) AS anzahl_kunden, APPROX_COUNT_DISTINCT(customer_id) AS
anzahl_eindeutig
FROM kunden
GROUP BY city;

SELECT customer_name, AVG(amount) AS durchschnittsbetrag, MIN(amount) AS min_betrag,
MAX(amount) AS max_betrag
FROM bestellungen
GROUP BY customer_name;
```

Für eine schnelle, explorative Übersicht über eine gesamte Tabelle eignen sich `describe()` und das umfangreichere `summary()`. `describe()` liefert je numerischer/String-Spalte `count`, `mean`, `stddev`, `min` und `max`. `summary()` erweitert dies standardmäßig um Perzentile (25 %, 50 %/Median, 75 %) und lässt sich über ein optionales Argument auf genau die gewünschten Statistiken einschränken.

```
kunden.describe().show()
bestellungen.summary().show()
# nur bestimmte Statistiken berechnen (spart Rechenzeit auf großen Tabellen):
bestellungen.summary("count", "min", "25%", "75%", "max").show()
```

Ergänzt aus der offiziellen PySpark-Dokumentation: `approx_count_distinct(col, rsd=0.05)` berechnet die approximative Anzahl unterschiedlicher Elemente einer Spalte über den HyperLogLog++-Algorithmus; der Parameter `rsd` steuert dabei den relativen Standardfehler – laut Dokumentation gilt: Ist `rsd < 0.01`, ist es effizienter, stattdessen `count_distinct()` (exakt) zu verwenden, da der Aufwand für sehr geringe Fehlertoleranzen den Vorteil der Approximation aufhebt. `dropDuplicates()` ohne Spaltenliste verhält sich laut Referenz identisch zu `distinct()`; mit Spaltenliste werden nur die angegebenen Spalten zur Duplikaterkennung herangezogen, wobei welche der doppelten Zeilen behalten wird, nicht deterministisch festgelegt ist.

Quelle: [pyspark.sql.functions.approx_count_distinct – Apache Spark Dokumentation](#)